Week 4 — Calibration & Baselines (Step-by-Step Manual)

```
# Week 4 — Calibration & Baselines (Step-by-Step Manual)

This week you'll turn your motion-tracking MVP into a **reliable** monitor by:
- establishing **per■fish baselines** (what "normal" looks like),
- tuning **noise filters** (bubbles, reflections, plants),
- setting **day/night aware alerts** that reduce false alarms,
- documenting a **stable configuration** you can reuse.

> Prereqs: Week 2 metrics flowing into InfluxDB (`fish_activity` with `distance_px`
per fish_id per minute) and a basic Grafana dashboard.

---

## ■ Objectives
1. Gather 3–7 days of data with consistent lighting and placement.
2. Quantify per■fish *normal* activity (median/percentiles per time-of-day).
3. Tune the motion pipeline (thresholds, masks) to minimize noise.
4. Implement **dynamic alerting**: deviations vs. each fish's own baseline.
5. Freeze a **v1.0 config** (values + Grafana alert rules).

---

## 0) Prep: Snapshot your current config
**Action:** Save current parameters so you can compare before/after.

- In your project folder (`~/fish-monitor`), copy the code as a dated snapshot:
```bash
cd ~/fish-monitor
cp motion_track_influx.py motion_track_influx.v1.preweek4.py
git init 2>/dev/null || true
git add . && git commit -m "Pre-Week4 snapshot"
```
Context: Easy rollback point before tuning begins.

---

1) Lock the scene (camera + lighting hygiene)
Action: Make the scene predictable so algorithm tuning lasts.

- Fix **camera position** and angle; re■autofocus once, then keep it.
- Use **one** lighting source (tank LED). Avoid ambient daylight swings.
- Turn off decorative air stones during calibration if they create bubbles in FOV.
- Wipe glass (inside/outside) and clean lens.

Context: Stable optics reduces mask/threshold tightness and false motion.

---

2) Add two quick filters in the code
Action: Reduce noise before retuning thresholds.

Open `motion_track_influx.py` and add:

- **ROI mask** (ignore static corners/surfaces that flicker):
```python
```
```

```
# After capturing a frame once, build a binary mask of the tank region only.
# Example (simple rectangle); refine to your tank shape if needed.
mask = np.zeros((FRAME_H, FRAME_W), dtype=np.uint8)
cv2.rectangle(mask, (40, 40), (FRAME_W-40, FRAME_H-40), 255, -1)  # shrink edges
# Later, apply mask to threshold image: th = cv2.bitwise_and(th, th, mask=mask)
```

- **Opening (erode->dilate) to kill speckles**:

```python
kernel = np.ones((3,3), np.uint8)
th = cv2.morphologyEx(th, cv2.MORPH_OPEN, kernel, iterations=1)
```

Also consider raising:

```python
MIN_CONTOUR_AREA = 300  # start 300–600 to ignore micro-bubbles
```

Context: You want fewer, larger blobs that map better to actual fish.

---

3) Choose working resolution & FPS

Action: Pick a sustainable processing rate.

- Start at **1280×720 @ 15–20 fps** in OpenCV:

```python
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 1280)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 720)
cap.set(cv2.CAP_PROP_FPS, 20)
```

- If CPU is spiky, drop to 960×540 or reduce FPS.

Context: Stable throughput keeps tracking IDs consistent.

---

4) Collect baseline data (3–7 days)

Action: Let the service run undisturbed.

- Ensure the systemd service is active:

```bash
sudo systemctl status fish-track
```

- Confirm points arrive in Influx:
  - Bucket `fish`, measurement `fish_activity` with tags `fish_id`, field `distance_px` (per minute).

Context: You'll use this data to compute per■fish baselines & time■of■day profiles.

---

5) Compute baselines (median & percentiles per fish, per hour)

Action: Derive robust baselines with Flux or Python. Below are two options.

Option A — Flux in Grafana (fast, no export)

Create a panel query (adjust range to last 7d):

```flux
import "experimental/aggregate"

fish = from(bucket: "fish")
  |> range(start: -7d)
  |> filter(fn: (r) => r._measurement == "fish_activity")
  |> filter(fn: (r) => r._field == "distance_px")

// Hour-of-day baseline per fish_id
fish
  |> map(fn: (r) => ({ r with hour: int(v: hour(t: r._time)) }))
  |> group(columns: ["fish_id","hour"])
  |> quantile(q: 0.5, method: "exact_selector")         // median
```

Duplicate with `q: 0.1` and `q: 0.9` for P10/P90 bands. Overlay median and bands per fish.

Option B — Python (export & crunch)

If you prefer CSV + Python (on your laptop or Pi):

- Export last 7d from Grafana Explore (as CSV).
- Use pandas to compute median/P10/P90 per fish_id + hour.
- Save a JSON with thresholds for reuse.

Context: Medians & percentile bands give you robust “normal” per fish and time■of■day.

---

6) Tune thresholds using baseline bands

Action: Convert baselines into alert thresholds.

- For each fish_id and hour-of-day:
  - **Low-activity threshold** = `max(P10, 0.3 * median)` as a guardrail.
  - **Severe inactivity** if distance_px ≈ 0 for **N** consecutive minutes (start with N=10).
- If tank has lights-off period, set **night mode**:
  - Lower expectations at night (e.g., allow values near 0 without alert).
  - Implement via Grafana “time range” or a query condition on hour-of-day.

Context: Dynamic thresholds cut false alarms and tailor to each fish’s rhythm.

---

7) Grafana panels & alerts (dynamic rules)

Action: Build a per■fish panel showing:

- last 24–72h `distance_px`
- overlay median & P10/P90 bands for that hour

Alert rule template: (adjust to your Grafana/Flux version)

- Condition: `distance_px` 10■min mean **< threshold(fish_id, hour)** for **10 min**
- Or: a **rolling z■score** alert where z < −2.0 for 10 min.

Flux helper for 10■min mean:

```flux
```

```
from(bucket: "fish")
  |> range(start: -24h)
  |> filter(fn: (r) => r._measurement == "fish_activity" and r._field == "distance_px" and r.fish_id == "3")
  |> aggregateWindow(every: 1m, fn: mean)
  |> aggregateWindow(every: 10m, fn: mean)
```

Context: Start conservative; you can tighten after observing a few days.

---

8) Validate with controlled tests

Action: Sanity■check alerts before trusting them.

- Temporarily **cover the camera** for 2–3 minutes → confirm no spurious “low activity” unless extended.
- **Gently net one fish** to reduce its movement for ~10–15 min → ensure alert triggers once, not flapping.
- Re-enable bubbles/decor **one at a time** and verify no false spikes.

Context: You want alerts that are **neither silent nor noisy**.

---

9) Freeze v1.0 configuration

Action: Persist your tuned values so they survive restarts and upgrades.

- In `motion_track_influx.py`, promote your final values:
  - `MIN_CONTOUR_AREA`, `MAX_ASSOC_DIST`
  - any **ROI mask** coordinates
  - chosen **FRAME_W/FRAME_H/FPS**
- Save a **config JSON** (for future AI phase) with your per■fish medians and thresholds.
- Commit to git:

```bash
git add .
git commit -m "Week4: tuned thresholds, masks, and baseline notes"
```

Context: A known■good state you can ship, restore, or compare later.

---

10) Optional refinements (nice wins)

- **Day/Night profiles:** separate baselines for light hours vs dark hours.
- **Smoothing:** exponential moving average (EMA) on distance to reduce jitter.
- **Zone metric:** count minutes near **top** vs **bottom** (future health signals).
- **Alert cool■downs:** avoid rapid repeat notifications.

■ Week 4 Checklist

- [ ] Scene stabilized (camera & lighting)
- [ ] ROI mask and morphological filtering applied
- [ ] Sustainable resolution/FPS chosen
- [ ] 3–7 days of data captured

- [ ] Median/P10/P90 baselines computed per fish & hour
- [ ] Dynamic alert thresholds configured in Grafana
- [ ] Alerts validated with controlled tests
- [ ] v1.0 config committed

Outcome: A robust, self■calibrated MVP with trustworthy alerts — ready for the AI upgrade in Phase 2.